

Justification of Technologies and Design Patterns

Why These Technologies?

For this Event Booking System prototype, I chose **Next.js (React)**, **TypeScript**, and **Vanilla CSS**.

1. **Next.js**: It allows for creating a full-stack application (both front-end UI and back-end API routes) within a single repository. This reduces boilerplate and eliminates the need to run two separate servers for the prototype.
2. **TypeScript**: Provides strong typing which is invaluable when implementing design patterns. Interfaces like ``Notifier`` or ``Event`` ensure that the factory and models adhere to strict contracts, avoiding runtime errors.
3. **Vanilla CSS**: Provides complete control over the visual presentation without adding the bloat of external styling libraries, making it easy to create a custom and robust UI.

Why These Patterns?

The core problem is managing event availability and user bookings seamlessly while keeping the codebase scalable.

1. **Singleton Pattern**:

- **Problem**: In a simulated backend without an external database, data is easily lost or recreated on every API request.
- **Justification**: By using the Singleton pattern for ``DatabaseConnection``, the application guarantees that all API requests share the exact same in-memory array of events and bookings. This prevents duplicate instances from resetting the event capacity, correctly simulating a real, centralized persistent store.

2. **Factory Pattern**:

- **Problem:** Users have different preferences for how they want to receive their booking confirmation (Email, SMS, Push). Writing conditional logic everywhere a notification is sent creates tight coupling.
- **Justification:** The ``NotifierFactory`` centralizes the object creation. If a new notification channel (e.g., WhatsApp) is added in the future, only the factory needs to be updated. The core business logic (``EventModel``) remains completely unchanged, adhering to the Open-Closed Principle.

3. **MVC (Model-View-Controller) Pattern:**

- **Problem:** Mixing data access, HTTP validation, and UI rendering in Next.js pages or API routes leads to spaghetti code that is hard to maintain or test.
- **Justification:** MVC cleanly separates the concerns.
 - The **Model** (``EventModel``) knows nothing about HTTP or UI; it just handles business rules.
 - The **Controller** (``BookingController``) deals with HTTP request validation and error mapping.
 - The **View** (React components) only renders what it is given and handles user clicks. This separation makes the application highly modular and allows frontend and backend logic to evolve independently.

Reflection: Closing Questions

1. Which of the three patterns did you find most intuitive, and which was hardest to justify? Why?

- **Most Intuitive:** The Singleton pattern was the most intuitive because its purpose is very straightforward: preventing the creation of multiple instances of a class. When dealing with something like a database connection or in-memory state, it immediately makes sense why having multiple copies would be problematic.

- **Hardest to Justify:** The MVC pattern, in the context of modern component-based frameworks like React, can sometimes feel hard to strictly justify because React naturally encourages grouping logic and UI within the same component file (hooks and JSX). Enforcing a strict traditional MVC structure requires discipline and separating concerns in ways that might feel slightly verbose for very small prototypes.

2. Think of a previous personal or school project. Would it have improved with one of these patterns? Which one, and why?

- In a previous group project where we built an e-commerce store, we struggled with managing multiple payment gateways (Stripe, PayPal, and Bank Transfer). Every time a user checked out, we had massive `if/else` blocks in our checkout route. That project would have massively improved using the **Factory Pattern**. A `PaymentProcessorFactory` could have abstracted the creation of the specific payment handler based on the user's choice, keeping our checkout logic clean and making it easy to add new payment methods later.

3. At what point could applying a pattern be counterproductive for a small project?

- Applying a pattern becomes counterproductive when it introduces unnecessary layers of abstraction that outweigh the complexity of the problem being solved. For example, if a small project only ever needs to send emails and will never scale to SMS or Push notifications, building a Factory pattern is "over-engineering". It adds extra files and interfaces that simply make the code harder to read for no tangible benefit.

4. If you had to explain the difference between Factory and MVC to a classmate who has never programmed, what analogy would you use?

- **Factory:** Imagine a car manufacturing plant. You walk in and say "I need a sports car," and the factory hands you one. You don't need to know how the engine was built or how the tires were attached; the factory handles the complex creation process and gives you the finished product.

- **MVC (Model-View-Controller):** Imagine a restaurant.

- The **View** is the menu and the dining area where the customer sits (what you see).

- The **Controller** is the waiter. You give your order (input) to the waiter, and the waiter takes it to the kitchen.

- The **Model** is the chef in the kitchen who knows the recipes and actually prepares the food (business logic/data). They all work together, but they have completely separate jobs.